

Projet

Un interpréteur pour un langage de dessin en 2D

Version du 3 avril 2026
Date limite : jeudi 7 mai

Consignes

Le projet doit être rendu avant le **jeudi 7 mai, à 23h59**. Le projet est à faire en binôme (pas de trinôme!). Les deux membres d’un binôme peuvent être dans des groupes de TP différents. Les monômes sont autorisés mais seront notés comme des binômes.

Rendu demandé. Le langage de programmation utilisé doit être OCaml. L’analyse lexicale doit être réalisée avec *ocamllex*, et l’analyse grammaticale avec *menhir*. Il est conseillé de vous servir des raccourcis de la bibliothèque standard de menhir. Pour la compilation, vous êtes libres de vous servir de *dune*, *ocamlbuild*, ou d’écrire un Makefile à la main.

Il n’est pas demandé de rédiger un rapport de projet ; cependant, on demande d’inclure dans le dépôt git de votre projet un fichier `ReadMe.txt` qui indique les commandes à lancer pour compiler et exécuter votre programme. Il est aussi conseillé d’inclure un sous-dossier **exemples** contenant un ou plusieurs fichiers à tester pour montrer les différentes fonctionnalités que vous aurez implémentées. De plus, il vous est demandé d’expliquer brièvement (dans le fichier `ReadMe.txt`) quelles sont les parties du sujet que vous avez réalisées, et comment vous avez réparti le travail dans le binôme.

Dépôt git. Le travail doit **obligatoirement** être réalisé via un dépôt git pour chaque groupe (binôme ou monôme). L’usage de ce gestionnaire de version permet d’éviter toute perte de donnée, et d’accéder si besoin à vos anciennes versions.

- Votre dépôt git doit être hébergé sur le serveur GitLab de l’UFR d’informatique. Pour cela, créez un *fork* (bifurcation) du dépôt git suivant, en choisissant la visibilité **privée** : <https://moule.informatique.univ-paris-diderot.fr/jledent/projet-gas6-2026>
- **Votre dépôt doit être rendu privé** dès sa création, avec accès uniquement aux membres du groupe. Tout code laissé en accès libre sur moule ou ailleurs sera considéré comme une incitation à la fraude, et sanctionné. Une fois votre dépôt git créé, ajoutez **tous** les enseignants du cours, en leur assignant le rôle Maintenir : @barszeza, @habermeh, @jledent, @treinen, @zvarin.
- Prenez soin de faire des **commits réguliers** (au *minimum* un pour chaque « Étape » du projet). Faites en sorte que les messages des commits soient clairs : nouvelle fonctionnalité, correction d’un bug, refactorisation du code, ...

Plagiat, code généré par IA. Il va de soi que votre travail doit être strictement personnel : aucune communication de code ou d’“idées” entre les groupes, aucune “aide” externe ou entre groupes. Nous vous rappelons que la fraude à un projet est aussi une fraude à un examen, passible de sanctions disciplinaires. L’utilisation de code généré par IA est **interdite** et pourra être sanctionnée au même titre que le plagiat.

Critères de notation. Le thème du cours étant l’analyse syntaxique, on accordera une attention toute particulière à la partie lexer/parser : pas de conflits dans la grammaire, des messages d’erreurs clairs et aussi précis que possibles en cas de problème de syntaxe dans le fichier d’entrée. Les projets seront évalués par une soutenance, qui comportera :

- Une brève démonstration du fonctionnement de votre projet.
- Des questions techniques sur le fonctionnement de votre code.

On ne demande pas de préparer une présentation, et on mettra l’accent sur les questions techniques. Tous les membres du binôme doivent être capables d’expliquer le code. Présenter un projet qui fonctionne sans savoir l’expliquer entraînera une mauvaise note.

En cas de difficultés. Pour toute question concernant le projet, une section « Questions Projet » est disponible sur Moodle (évidemment, n’y postez pas de bout de code). Vous pouvez aussi poser des questions directement à votre chargé-e de TD, ou par mail en ajoutant le tag “[GAS6]” dans l’objet du mail. N’hésitez pas à signaler tout problème technique (problèmes d’installation, de compilation) **dès que possible** ! C’est-à-dire, pas la veille du rendu et encore moins le jour de la soutenance.

0 Objectif

Dans ce projet vous allez concevoir et implémenter un interpréteur pour un langage de dessin dans le plan. Cet interpréteur prend en entrée un fichier texte qui décrit une suite d’instructions. En exécutant ces instructions, il génère une image à l’écran. Le principe repose sur un curseur virtuel qui se déplace sur l’écran. Ce curseur peut être dans deux états possibles. En position basse (pinceau baissé), chaque déplacement trace un trait à l’écran ; en position haute (pinceau levé), les déplacements ne dessinent rien. On peut déplacer le curseur à l’aide des deux instructions suivantes :

- **Avancer** `<distance>` : le curseur avance d’une distance donnée dans la direction actuelle.
- **Tourner** `<angle>` : le curseur reste à sa position actuelle, mais change de direction en tournant d’un angle donné.

Les distances (en pixels) et les angles (en degrés) sont donnés par des expressions, qui supportent les opérateurs $+$, $-$, $\times$, $/$, en respectant la priorité des opérations habituelle.

Le projet est structuré en plusieurs étapes de difficulté croissante. La première étape consiste à implémenter un interpréteur pour une version très minimaliste du langage. Puis, chaque étape viendra rajouter de nouvelles fonctionnalités. Il est **fortement recommandé** de bien tester votre programme à la fin de l’étape N pour s’assurer que tout fonctionne correctement avant de passer à l’étape $N + 1$. Il est également avisé de faire un commit sur votre dépôt git à la fin de chaque étape, pour conserver une version fonctionnelle du projet avant d’entamer des modifications.

1 Première Étape

Dans un premier temps, écrivez un analyseur lexical et syntaxique afin de construire l'arbre de syntaxe abstraite d'un programme décrit par la grammaire ci-dessous. Ensuite, écrivez un interpréteur qui affichera le dessin généré par le programme donné en entrée. Pour l'affichage, vous utiliserez la bibliothèque **Graphics**¹ d'OCaml.

État initial. Au lancement du programme, on ouvre une fenêtre graphique de taille 800 × 800, le curseur est placé au centre du canevas graphique, en orientation nord (c'est-à-dire, vers le haut), et le pinceau est levé.

1.1 La grammaire

La grammaire du langage est donnée ci-dessous sous forme de Backus-Naur. Les non-terminaux sont donnés entre *<chevrons>*, les terminaux (jetons) en police **code**. Les symboles ; + - * / () sont également des terminaux. Le symbole ε désigne le mot vide, et EOF désigne la fin du fichier. Il n'est pas nécessaire de conserver la structure exacte de la grammaire : vous pouvez (c'est même recommandé) utiliser les fonctions de la bibliothèque standard de Menhir² pour la simplifier.

<i><programme></i>	::= <i><bloc_instructions></i> EOF
<i><bloc_instructions></i>	::= <i><instruction></i> ; <i><bloc_instructions></i> ε
<i><instruction></i>	::= BaisserPinceau LeverPinceau Avancer <i><expression></i> Tourner <i><expression></i>
<i><expression></i>	::= nombre (<i><expression></i>) <i><expression></i> <i><opérateur></i> <i><expression></i>
<i><opérateur></i>	::= + - * /

où le jeton **nombre** désigne une suite non-vide de chiffres, possiblement précédée par un -.

Exemple. Par exemple, le programme ci-dessous dessine un carré de 10 × 10 pixels :

```
BaisserPinceau;  
Avancer 10;  
Tourner 90;  
Avancer 10;  
Tourner 90;  
Avancer 10;  
Tourner 90;  
Avancer 10;
```

1. Voir <https://ocaml.org/manual/4.07/libref/Graphics.html>

2. Voir <https://gallium.inria.fr/~fpottier/menhir/manual.html#sec38>

1.2 Gestion d'erreurs

Votre programme doit traiter proprement toutes les erreurs qui peuvent se produire :

- Erreurs de syntaxe : le programme donné en entrée est mal formé et l'analyse lexicale ou syntaxique a échoué.
- Erreurs pendant l'exécution : le curseur sort du canevas ; division par 0.

Dans chaque cas, il faudra afficher un message le plus explicite possible, détaillant l'erreur rencontrée, éventuellement le numéro de ligne où elle s'est produite, et puis arrêter le programme.

2 Deuxième Étape

Ajouter des instructions permettant de modifier la couleur et la largeur du trait. Initialement, le pinceau est noir, et la largeur est de 1 pixel.

```
<instruction> ::= ...  
                | CouleurPinceau <couleur>  
                | LargeurPinceau <expression>  
<couleur>      ::= noir | rouge | bleu | vert | jaune | hexcode
```

où la couleur peut être donnée par une constante **noir**, **rouge**, **bleu**, **vert**, **jaune**, ou bien en donnant le code hexadécimal de la couleur voulue.

Exemple. Par exemple, le code ci-dessous dessine un triangle tricolore.

```
CouleurPinceau vert;  
LargeurPinceau 5;  
BaisserPinceau;  
Avancer 50;  
Tourner 120;  
CouleurPinceau bleu;  
Avancer 50;  
Tourner 120;  
CouleurPinceau FF1493;  
Avancer 50;
```

3 Troisième Étape

On souhaite désormais étendre la syntaxe du langage pour permettre de stocker des valeurs numériques dans des variables, qui peuvent être modifiées au cours de l'exécution du programme. Chaque variable devra être *déclarée* à l'aide du mot-clé **Var** avant d'être utilisée. Il faudra veiller à ce que toute variable utilisée soit bien déclarée, et que chaque variable soit déclarée une seule fois. Il est possible de déclarer une variable sans l'initialiser ; il faudra veiller à ce qu'elle soit initialisée avant de l'utiliser.

On ajoute à notre grammaire les instructions suivantes :

```
<instruction> ::= ...  
                | Var id  
                | Var id = <expression>  
                | id = <expression>
```

où le jeton `id` désigne une suite non-vide de caractères alphanumériques, dont le premier caractère doit être une lettre (minuscule ou majuscule).

Exemple. Le code ci-dessous devrait dessiner un carré de côté 10.

```
Var angleDroit = 90;
Var longueur = 10;
BaisserPinceau;
Avancer longueur;
Tourner angleDroit;
Avancer longueur;
Tourner angleDroit;
Avancer longueur;
Tourner angleDroit;
Avancer longueur;
```

4 Quatrième Étape

Ajoutez des boucles et des tests, avec la syntaxe suivante :

Si *<condition>* **Alors** *<instruction>*

Si *<condition>* **Alors** *<instruction>* **Sinon** *<instruction>*

Tant que *<condition>* **Faire** *<instruction>*

Repeter *<expression>* **Fois** *<instruction>*

Il faudra donc introduire un non-terminal *<condition>* qui permet de tester une condition booléenne : les constantes **vrai**, **faux**, les opérateurs de comparaison `==`, `!=`, `<`, `<=`, `>`, `>=`, et les opérateurs booléens `&&`, `||`, `!`.

Si on veut pouvoir exécuter plusieurs instructions dans une boucle, il faut une syntaxe pour délimiter le bloc d'instructions concerné. On ajoute aussi à la grammaire la règle :

<instruction> ::= ... | **Debut** *<bloc_instructions>* **Fin**

Exemple. Vous devriez maintenant pouvoir interpréter les programmes suivants :

```
BaisserPinceau;
Var distance = 200;
Tant que distance > 0 Faire
Debut
    Avancer distance;
    Tourner 90;
    distance = distance - 10;
Fin;
```

```
BaisserPinceau;
Repeter 10 Fois
Debut
    Tourner 36;
    Repeter 5 Fois
    Debut
        Avancer 108;
        Tourner 72;
    Fin;
Fin;
```

Indications. Attention aux conflits dans la grammaire. Le problème du « if ... else » a été vu en CM. Attention aussi à la portée des variables : une variable déclarée dans une boucle ne doit plus exister une fois que l'on est sorti de la boucle.

5 Bonus (facultatif)

Si vous souhaitez aller plus loin, voici quelques idées de fonctionnalités supplémentaires à implémenter. Avant d'entamer les questions bonus, assurez-vous d'avoir poussé sur Git une version fonctionnelle du projet jusqu'à l'étape 4, et faites les bonus dans une nouvelle branche.

1. Autoriser le signe moins unaire dans les expressions, pour permettre par exemple l'instruction « **Tourner -angle** ».
2. Permettre que des variables puissent stocker des valeurs booléennes ou des couleurs.
3. Ajouter la possibilité de tirer aléatoirement un nombre ou une couleur.
4. Ajouter des boucles « For » avec initialisation d'un compteur de boucle.
5. Permettre à l'utilisateur de définir des fonctions/procédures, qui peuvent prendre en entrée des arguments, et produisent un dessin. Pour vérifier que tout fonctionne, essayez de dessiner une fractale de votre choix (flocon de von Koch, triangle de Sierpiński, courbe du dragon, courbe de Hilbert, ...)
6. Un mode interactif où l'utilisateur entre des commandes et voit au fur et à mesure le résultat de ces commandes dans une fenêtre graphique